

Reference Notes

Week 11: Dictionaries - Key-Value Magic

7th Grade Computer Science - Module 2

1 Introduction

1.1 This Week's Big Question

How do different data structures help us organize and manipulate information? This week, we explore dictionaries - a powerful new way to store data that lets us look things up by name instead of by position!

Prerequisites

Before starting this week, you should be comfortable with:

- Creating and modifying lists
- Understanding list indexing and methods
- Working with nested data structures (2D lists)

1.2 What You Already Know

You've mastered lists, which store items in order and let you access them by position (index 0, 1, 2...). You know how to search through lists, modify them, and even create lists of lists. But what if you want to look something up by name instead of remembering its position?

1.3 What You'll Be Able to Do

- Store information as key-value pairs (like a real dictionary!)
- Look up data instantly using meaningful names
- Count occurrences and track frequencies efficiently
- Build programs like contact lists, game scores, and inventory systems
- Choose the right data structure for your problem

2 VIDEO 1: Dictionary Fundamentals

2.1 Understanding Dictionaries

Imagine you have a phone book where you look up people by their names to find their phone numbers. That's exactly how Python dictionaries work! Instead of remembering that "Ali's number is at position 3 in my list," you can just ask for the number associated with "Ali". Dictionaries store information as pairs: a key (like the name) and a value (like the phone number).

2.2 Your First Dictionary

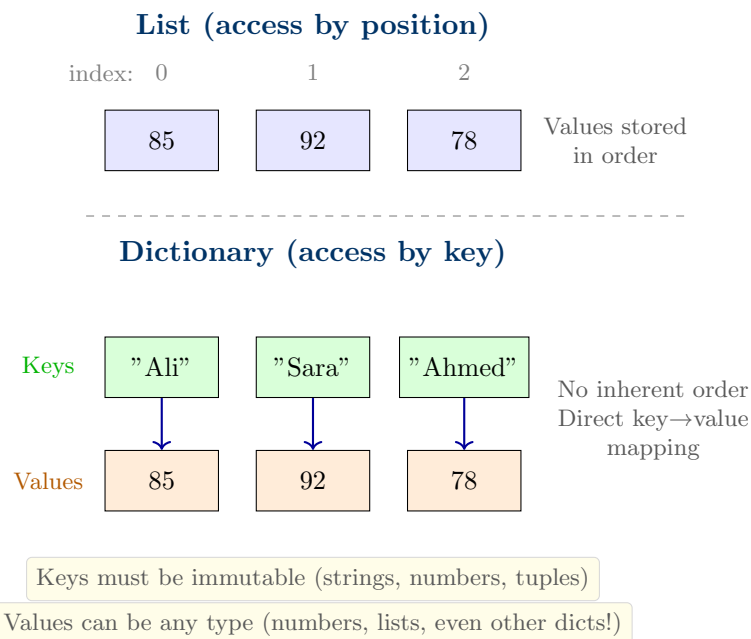
Let's create a simple dictionary to store student grades. Notice how we use curly braces {} and pair each name with a grade using a colon. This is much more intuitive than trying to remember which index corresponds to which student!

```
1 # Creating a dictionary of student grades
2 grades = {"Ali": 85, "Sara": 92, "Ahmed": 78}
3 print(grades) # {'Ali': 85, 'Sara': 92, 'Ahmed': 78}
4
5 # Accessing a value by its key
6 ali_grade = grades["Ali"]
7 print(f"Ali's grade: {ali_grade}") # Ali's grade: 85
```

See how we accessed Ali's grade directly by using his name as the key? No need to remember index positions!

2.3 Dictionary Structure Visualization

To truly understand dictionaries, let's visualize how they store data differently from lists. In a list, items are lined up in order, but in a dictionary, each key points directly to its value - there's no inherent ordering.



Notice how the dictionary doesn't care about order - it cares about the association between keys and their values. This makes lookups incredibly fast!

2.4 Adding and Updating Dictionary Items

Dictionaries are mutable like lists, which means we can add new items or change existing ones. Let's see how we can build up a dictionary over time and modify values when needed.

```

1 # Start with an empty dictionary
2 inventory = {}
3
4 # Add items one by one
5 inventory["apples"] = 5
6 inventory["bananas"] = 3
7 inventory["oranges"] = 7
8
9 # Update an existing value
10 inventory["apples"] = 10 # We got more apples!
11
12 # Add a new item
13 inventory["mangoes"] = 2
14
15 print(inventory)
16 # {'apples': 10, 'bananas': 3, 'oranges': 7, 'mangoes': 2}

```

See how flexible this is? We can start with an empty dictionary and build it up as we get new information!

⚠ Common Error

Common Mistake: Trying to access a key that doesn't exist causes a `KeyError`!

```

1 grades = {"Ali": 85, "Sara": 92}
2 print(grades["Hassan"]) # KeyError: 'Hassan'
3
4 # Better: Use .get() with a default value
5 print(grades.get("Hassan", 0)) # Returns 0 if not found

```

👉 Quick Check

What will this code print?

```

1 scores = {"player1": 100, "player2": 150}
2 scores["player1"] = 120
3 scores["player3"] = 80
4 print(len(scores))

```

Think about it, then check: It prints 3 (three key-value pairs in the dictionary)

3 VIDEO 2: Working with Keys and Values

3.1 Understanding Keys and Values

In a dictionary, keys are like labels or names that help us find our data, while values are the actual data we're storing. Keys must be unique (you can't have two students with the exact same name in your gradebook), but values can repeat (multiple students can have the same grade).

3.2 Dictionary Methods in Action

Python gives us powerful methods to work with dictionary keys and values separately or together. Let's explore these methods with a simple example of tracking game high scores.

```

1 # Game high scores dictionary
2 high_scores = {"Mario": 5000, "Luigi": 4500, "Peach": 5500}
3
4 # Get all keys (player names)
5 players = high_scores.keys()
6 print(list(players)) # ['Mario', 'Luigi', 'Peach']
7
8 # Get all values (scores)
9 scores = high_scores.values()
10 print(list(scores)) # [5000, 4500, 5500]
11
12 # Get all key-value pairs
13 all_data = high_scores.items()
14 print(list(all_data))
15 # [('Mario', 5000), ('Luigi', 4500), ('Peach', 5500)]

```

These methods are incredibly useful when you need to process all the data in your dictionary!

3.3 Checking for Keys

Before accessing a dictionary value, it's often smart to check if the key exists. Python makes this easy with the `in` operator. This helps us avoid errors and write more robust programs.

```

1 contact_list = {"Ali": "0300-1234567", "Sara": "0321-9876543"}
2
3 # Check if a contact exists
4 if "Ali" in contact_list:
5     print(f"Ali's number: {contact_list['Ali']}")
6 else:
7     print("Ali not found in contacts")
8
9 # Check if someone is NOT in the dictionary
10 if "Hassan" not in contact_list:
11     print("Hassan's number not saved yet")
12     contact_list["Hassan"] = input("Enter Hassan's number: ")

```

This pattern of checking before accessing prevents your program from crashing with a `KeyError`!

Tip

Use `dict.get(key, default)` when you want a default value for missing keys. Use `key in dict` when you need to know if a key exists before doing something special.

Quick Check

Given this dictionary: `ages = {"Ali": 13, "Sara": 14}`
Which of these will cause an error?

1. `ages.get("Hassan", 12)`
2. `ages["Hassan"]`
3. `"Hassan" in ages`

Answer: Only option 2 will cause a `KeyError`!

4 VIDEO 3: Common Dictionary Patterns

4.1 Frequency Counting Pattern

One of the most powerful uses of dictionaries is counting how often things appear. This pattern appears everywhere - from analyzing text to tracking game statistics. Let's see how dictionaries make this incredibly simple!

4.2 Building a Frequency Counter

Imagine you want to count how many times each letter appears in a word. With a dictionary, we can build this counter step by step, creating new entries as we discover new letters.

```

1 # Count letter frequencies
2 word = "banana"
3 letter_count = {}
4
5 for letter in word:
6     if letter in letter_count:
7         letter_count[letter] += 1 # Increment existing count
8     else:
9         letter_count[letter] = 1 # First time seeing this letter
10
11 print(letter_count) # {'b': 1, 'a': 3, 'n': 2}
12
13 # Shorter way using .get()
14 letter_count2 = {}
15 for letter in word:
16     letter_count2[letter] = letter_count2.get(letter, 0) + 1
    
```

The second approach using `.get()` is more elegant - it returns 0 if the letter isn't found, then adds 1!

4.3 Grouping Data by Category

Dictionaries excel at organizing data into categories. Let's say you're organizing students by their favorite subjects - a dictionary makes this natural and efficient.

```

1 # Group students by favorite subject
2 students_by_subject = {}
3
4 # Add students to their subject groups
5 def add_student(name, subject):
6     if subject not in students_by_subject:
    
```

```

7         students_by_subject[subject] = [] # Create new list
8         students_by_subject[subject].append(name)
9
10    add_student("Ali", "Math")
11    add_student("Sara", "Science")
12    add_student("Ahmed", "Math")
13    add_student("Fatima", "Science")
14
15    print(students_by_subject)
16    # {'Math': ['Ali', 'Ahmed'], 'Science': ['Sara', 'Fatima']}
```

See how we're using a dictionary of lists? Each subject (key) maps to a list of students (value)!

💡 Rule

Dictionaries are perfect when you need to:

- Look up data by name/ID rather than position
- Count occurrences of items
- Group related data together
- Cache/store results for quick access

👉 Quick Check

Write code to count how many times each number appears in this list:

```
numbers = [1, 2, 1, 3, 2, 1, 3, 3, 3]
```

Hint: Use the frequency counter pattern!

5 VIDEO 4: When to Choose Dictionaries

5.1 Comparing Lists vs Dictionaries

Now that you know both lists and dictionaries, how do you choose which one to use? The answer depends on how you need to access and organize your data. Let's explore when each data structure shines.

5.2 Decision Guide for Data Structures

Different problems need different tools. Here's a practical comparison to help you make the right choice for your programs.

Scenario	Best Choice	Why
Store test scores in order	List	Order matters, access by position
Phone book	Dictionary	Look up by name, not position
Game leaderboard (top 10)	List	Order is crucial, limited size
Student ID to name mapping	Dictionary	Direct lookup by ID
Shopping cart items	List	Order of adding might matter
Word count in a paragraph	Dictionary	Count frequency by word

5.3 Real Program Comparison

Let's see the same problem solved with both data structures to understand the practical difference. We'll build a simple grade lookup system.

Using Lists (Harder)

```

1 names = ["Ali", "Sara", "Ahmed"]
2 grades = [85, 92, 78]
3
4 # Find Ali's grade
5 name_to_find = "Ali"
6 for i in range(len(names)):
7     if names[i] == name_to_find:
8         print(grades[i])
9         break
10 # Need to search every time!
```

Using Dictionary (Easier)

```

1 grades = {"Ali": 85,
2           "Sara": 92,
3           "Ahmed": 78}
4
5 # Find Ali's grade
6 name_to_find = "Ali"
7 if name_to_find in grades:
8     print(grades[name_to_find])
9
10 # Direct access, no loop needed!
```

The dictionary version is cleaner, faster, and less error-prone. No need to keep two lists synchronized!

5.4 Building a Complete Program

Let's combine everything we've learned to build a simple but complete contact management system. This shows how dictionaries make real programs easier to write and understand.

```

1 # Simple contact manager
2 contacts = {}
3
4 def add_contact(name, number):
5     contacts[name] = number
6     print(f"Added {name}")
7
8 def find_contact(name):
9     return contacts.get(name, "Not found")
10
11 def show_all():
12     for name, number in contacts.items():
13         print(f"{name}: {number}")
14
15 # Use the system
16 add_contact("Mom", "0300-1111111")
17 add_contact("School", "042-3333333")
18 print(find_contact("Mom")) # 0300-1111111
19 show_all() # Shows all contacts
```

This program would be much more complex with lists - you'd need to search through the entire list each time!

👉 Quick Check

You're making a game where players collect different types of gems. Should you use a list or dictionary to track how many of each gem type a player has? Why?
 Answer: Dictionary! You want to look up counts by gem type (key) not position.

6 VIDEO 5: Summary

6.1 What We Learned This Week

This week, you discovered a powerful new way to organize data - dictionaries! You learned that dictionaries store information as key-value pairs, making it incredibly easy to look things up by name rather than position. You mastered adding, updating, and accessing dictionary items, and most importantly, you learned when to choose dictionaries over lists.

You can now build programs that efficiently count things, organize data by categories, and create lookup systems like contact lists or game inventories. The frequency counter pattern you learned is used everywhere in real programming, from analyzing text to tracking website visitors. You've also developed the judgment to choose the right data structure for your problem - a crucial programming skill!

★ Landmark Moment

You now understand a fundamental computer science concept: the hash table (what Python calls a dictionary)! This data structure powers everything from database indexes to web caching. When websites remember your preferences or games track your achievements, they're using dictionaries behind the scenes!

7 Quick Reference

📖 Quick Reference

Week 11 Essential Patterns:

Dictionary Basics:

- Create: `dict = {"key": value, "key2": value2}`
- Access safely: `dict.get("key", default)`
- Add/Update: `dict["key"] = new_value`
- Check existence: `if "key" in dict:`

Dictionary Methods:

- All keys: `dict.keys()`
- All values: `dict.values()`
- All pairs: `dict.items()`
- Remove: `dict.pop("key")` or `del dict["key"]`

Common Patterns:

- Frequency counter: `count[item] = count.get(item, 0) + 1`
- Grouping: Create dict of lists for categories
- Safe access: Always use `.get()` or check with `in`
- Common mistake: `KeyError` from missing keys

7.1 Reflection Questions

1. What was the trickiest part about understanding key-value pairs? How did you overcome it?
2. Can you think of three real-world things that work like dictionaries (hint: anything with lookup)?
3. When debugging dictionary KeyErrors, what strategy worked best for you?
4. How do you feel about having multiple data structures to choose from? Empowering or confusing?
5. Design a dictionary structure for your favorite game or app - what would be the keys and values?

Next week: We'll learn about functions - how to package our code into reusable building blocks that make our programs more organized and powerful!